

LINUX LAB

C PROGRAMMING

REPORT

38 'AI'

Prepared By:

Rupesh Yadav

Presented To:

Rupak Rajbanshi



I. Introduction.....	3
II. C Program Structure and Examples	3
A. Basic Hello World Program.....	3
B. Program with User Input.....	3
C. Program with Formatted Output	4
III. GCC Compilation Process	5
A. Installing GCC on Linux.....	5
B. Four Stages of Compilation	5
C. Compilation Stage Examples	6
1) Stage 1: Preprocessing	6
2) Stage 2: Compilation to Assembly.....	6
3) Stage 3: Assembly to Object File.....	7
4) Stage 4: Linking.....	7
IV. Return Values and Exit Status	8
A. Understanding Return Values	8
B. Checking Exit Status	8
V. Knowledge Check Questions	9
A. Question 1	9
B. Question 2	9
C. Question 3	9
D. Question 4	9
E. Question 5	10
F. Question 6.....	10
VI. Lab Tasks Completed	10
1) Task 1	10
2) Task 2	10
3) Task 3	11
4) Task 4	11
5) Task 5	12
VII. Conclusion	12
GitHub Repository	13

I. Introduction

This report documents the practical work completed for Lab 2 of ST5003CMD Operating Systems, Security and Networks. The focus is on understanding C program structure, compilation stages, and basic Linux commands.

II. C Program Structure and Examples

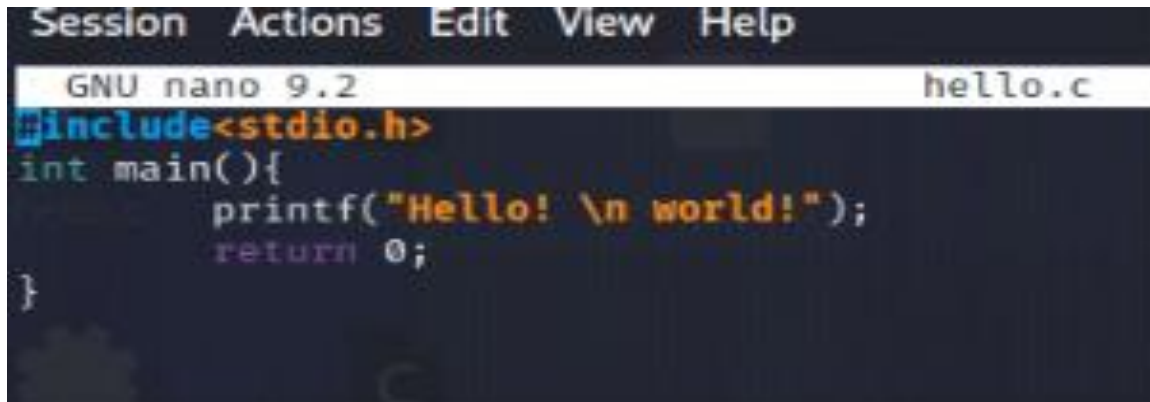
A. Basic Hello World Program

The following demonstrates a basic C program structure with preprocessor directives, main function, and return statement:

```
#include <stdio.h>

int main() {
    printf("Hello, World!\n");
    return 0;
}
```

Listing 1. Basic Hello World Program



B. Program with User Input

This example demonstrates the use of printf and scanf for input/output operations:

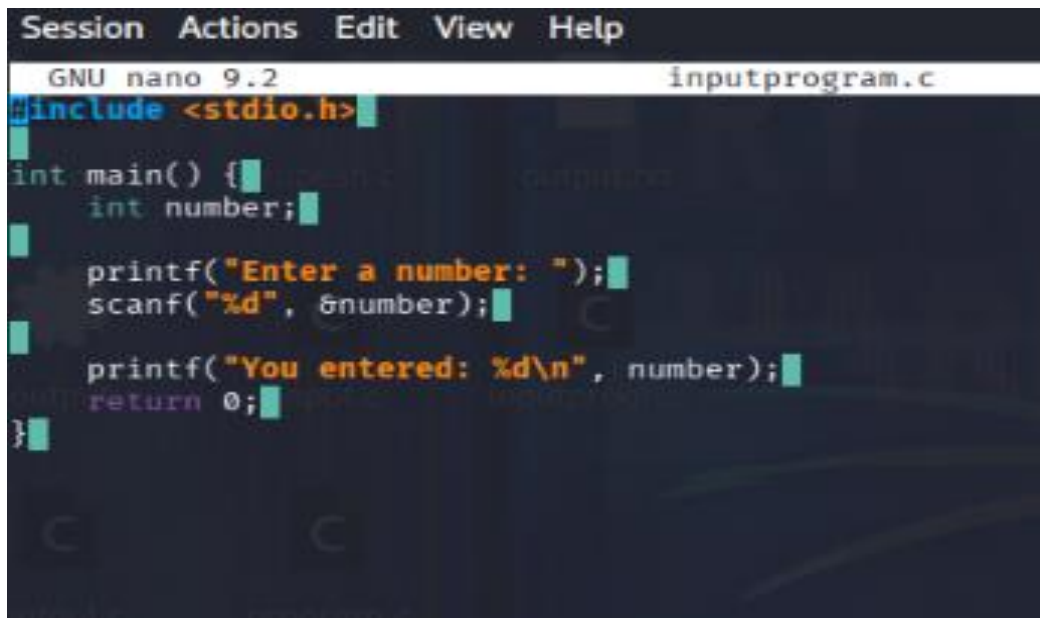
```
#include <stdio.h>

int main() {
    int number;

    printf("Enter a number: ");
    scanf("%d", &number);

    printf("You entered: %d\n", number);
    return 0;
}
```

Listing 2. Program with User Input



```
Session Actions Edit View Help
GNU nano 9.2 inputprogram.c
#include <stdio.h>

int main() {
    int number;

    printf("Enter a number: ");
    scanf("%d", &number);

    printf("You entered: %d\n", number);
    return 0;
}
```

C. Program with Formatted Output

Demonstrating multiple format specifiers and variables:

```
#include <stdio.h>

int main() {
    char name[50];
    int age;
    float height;

    printf("Enter your name: ");
    scanf("%s", name);

    printf("Enter your age: ");
    scanf("%d", &age);

    printf("Enter your height (m): ");
    scanf("%f", &height);

    printf("\n--- User Info ---\n");
    printf("Name: %s\n", name);
    printf("Age: %d years\n", age);
    printf("Height: %.2f m\n", height);

    return 0;
}
```

Listing 3. Program with Formatted Output

```
Session Actions Edit View Help
GNU nano 9.2 formattedoutput.c
#include <stdio.h>

int main() {
    char name[50];
    int age;
    float height;

    printf("Enter your name: ");
    scanf("%s", name);

    printf("Enter your age: ");
    scanf("%d", &age);

    printf("Enter your height (m): ");
    scanf("%f", &height);

    printf("\n--- User Info ---\n");
    printf("Name: %s\n", name);
    printf("Age: %d years\n", age);
    printf("Height: %.2f m\n", height);

    return 0;
}
```

III. GCC Compilation Process

A. Installing GCC on Linux

The following commands are used to install and verify GCC:

```
# Install build-essential package
sudo apt install build-essential

# Check GCC version
gcc --version
```

Listing 4. GCC Installation Commands

```
(kali@kali)-[~/Desktop]
└─$ sudo apt install build-essential
[sudo] password for kali:
build-essential is already the newest version (12.12).
kSummary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1236

(kali@kali)-[~/Desktop]
└─$ gcc --version
gcc (Debian 15.2.0-17) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

B. Four Stages of Compilation

The C compilation process consists of four distinct stages. Each stage transforms the source code closer to an executable program.

Stage	Command	Output	Purpose
1. Preprocessing	<code>gcc -E hello.c -o hello.i</code>	.i file	Expand headers & macros
2. Compiling	<code>gcc -S hello.i -o hello.s</code>	.s file	Generate Assembly code
3. Assembling	<code>gcc -c hello.s -o hello.o</code>	.o file	Create object file
4. Linking	<code>gcc hello.o -o hello</code>	executable	Final executable

Table I. The Four Stages of C Compilation

C. Compilation Stage Examples

1) Stage 1: Preprocessing

```
# Expand all headers and macros
gcc -E hello.c -o hello.i

# View the preprocessed output
cat hello.i | less
```

Listing 5. Preprocessing Stage

```

(kali@kali)-[~/Desktop]
$ gcc --version
gcc (Debian 15.2.0-17) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

(kali@kali)-[~/Desktop]
$ gcc -E hello.c -o hello.i

(kali@kali)-[~/Desktop]
$ cat hello.i | less

```

2) Stage 2: Compilation to Assembly

```

# Generate assembly code
gcc -S hello.i -o hello.s

# View assembly code
cat hello.s

```

Listing 6. Compilation Stage

```

Session  Actions  Edit  View  Help
(kali@kali)-[~/Desktop]
$ gcc -S hello.i -o hello.s
(kali@kali)-[~/Desktop]
$ cat hello.s
.file "hello.c"
.text
.section .rodata
.LC0:
.string "Hello! \n world!"
.text
.globl main
.type main, @function
main:
.LFB0:
.cfi_startproc
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
leaq .LC0(%rip), %rax
movq %rax, %rdi
movl $0, %eax
call printf@PLT
movl $0, %eax
popq %rbp
.cfi_def_cfa 7, 8

```

3) Stage 3: Assembly to Object File

```

# Create object file (machine code)
gcc -c hello.s -o hello.o

```

```
# Check file type
file hello.o
```

Listing 7. Assembly Stage

```
(kali@kali)-[~/Desktop]
$ gcc -c hello.s -o hello.o

(kali@kali)-[~/Desktop]
$ file hello.c
hello.c: C source, ASCII text

(kali@kali)-[~/Desktop]
$
```

4) Stage 4: Linking

```
# Create final executable
gcc hello.o -o hello

# Run the program
./hello
```

Listing 8. Linking Stage

```
(kali@kali)-[~/Desktop]
$ gcc hello.o -o hello

(kali@kali)-[~/Desktop]
$ ./hello
Hello!
world!
```

IV. Return Values and Exit Status

A. Understanding Return Values

The return value of `main()` indicates program success or failure to the OS:

```
#include <stdio.h>

int main() {
    printf("Program executed successfully\n");
    return 0; // 0 indicates success
}
```

Listing 9. Program with Return Value

```
(kali@kali)-[~/Desktop]
$ nano returnvalue.c
```

```
Session Actions Edit View Help
GNU nano 9.2 returnvalue.c *
#include <stdio.h>

int main() {
    printf("Program executed successfully\n");
    return 0; // 0 indicates success
}
```

B. Checking Exit Status

```
# Run program
./hello

# Check exit status (returns 0 for success)
echo $?

# Try with return 1
# Modify program to return 1, recompile, and check:
echo $? # Should return 1 (indicating error)
```

Listing 10. Checking Program Exit Status

```
(kali@kali)-[~/Desktop]
$ ./hello
Hello!
world!

(kali@kali)-[~/Desktop]
$ echo $
$
```

V. Knowledge Check Questions

The following questions assess understanding of Lecture 2 concepts.

A. Question 1

Q: What is the purpose of the `#include` directive in C programming? Provide an example of its usage.

A: The `#include` directive tells the preprocessor to copy the entire contents of a specified header file into the source file before compilation starts. It gives a program access to functions, macros, and declarations defined elsewhere, so

they do not need to be rewritten. System headers are included in angle brackets, while user-created headers use quotes. For example, `#include <stdio.h>` provides access to input/output functions such as `printf()` and `scanf()`.

B. Question 2

Q: Explain the difference between the four stages of compilation. What file extension is produced at each stage?

A: C compilation happens in four sequential stages, each moving the code closer to a runnable program. Preprocessing expands headers and macros, producing a `.i` file. Compiling translates that preprocessed code into assembly, producing a `.s` file. Assembling converts the assembly into machine code, producing a `.o` object file, which is finally linked with any needed libraries into the finished executable with no file extension.

C. Question 3

Q: What does a return value of 0 from the `main()` function indicate to the operating system? What about a non-zero return value?

A: A return value of 0 tells the operating system that the program ran and finished successfully with no errors. A non-zero return value signals that something went wrong during execution, and different numbers can represent different specific error conditions. This value becomes the program's exit status, which can be checked immediately afterward using the shell command `echo $?`.

D. Question 4

Q: Write a C program that declares an integer variable, asks the user to input a value, and displays the entered value back to the user.

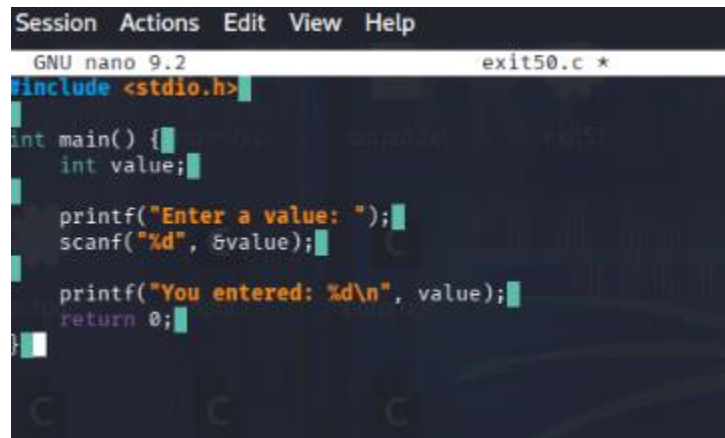
```
#include <stdio.h>

int main() {
    int value;

    printf("Enter a value: ");
    scanf("%d", &value);

    printf("You entered: %d\n", value);
    return 0;
}
```

A: The program declares an integer variable, prompts the user with `printf()`, and reads their input using `scanf()` with the `&` operator to store it directly into that variable's memory address. It then prints the stored value back out using another `printf()` with the `%d` format specifier.





E. Question 5

Q: What GCC command would you use to: (1) only preprocess a file without compiling, (2) generate assembly code but not assemble it, (3) create an object file without linking?

A: To only preprocess, use `gcc -E hello.c -o hello.i`, which just expands headers and macros. To stop after generating assembly, use `gcc -S hello.i -o hello.s`. To create an object file without linking it into a full executable, use `gcc -c hello.s -o hello.o`. Each flag (-E, -S, -c) tells GCC to stop the pipeline at that specific stage.

F. Question 6

Q: Explain why C is called a "compiled language" and how this differs from interpreted languages.

A: C is called a compiled language because its entire source code is translated into machine code by a compiler before the program ever runs, producing a standalone executable file. Interpreted languages, by contrast, are read and executed line-by-line at runtime by an interpreter, without ever producing a separate executable file. Compiled programs typically run faster since translation happens once ahead of time, while interpreted programs run slower but are often quicker to test and debug. Python and JavaScript are common examples of interpreted languages.

VI. Lab Tasks Completed

1) Task 1

Compiled `hello.c` with `return 0` and verified exit status using `echo $?`.

```
Session Actions Edit View Help
GNU nano 9.2 hello.c
#include<stdio.h>
int main(){
    printf("Hello! \n world!");
    return 0;
}
```

2) Task 2

Modified the program to `return 1` and observed a different exit status.

```
Session Actions Edit View Help
GNU nano 9.2 hello.c *
#include<stdio.h>
int main(){
    printf("Hello! \n world!");
    return 1;
}
```

3) Task 3

Created a program accepting user input and displaying it back.

```
Session Actions Edit View Help
GNU nano 9.2 inputprogram.c
#include <stdio.h>
int main() {
    int number;
    printf("Enter a number: ");
    scanf("%d", &number);
    printf("You entered: %d\n", number);
    return 0;
}
```

```
(kali@kali)-[~/Desktop]
$ ./inputprogram.c
zsh: permission denied: ./inputprogram.c

(kali@kali)-[~/Desktop]
$ gcc inputprogram.c -o inputfile

(kali@kali)-[~/Desktop]
$ ./inputfile
Enter a number: 34
You entered: 34
```

4) Task 4

Executed all four compilation stages separately and documented the output files at each step.

```

(kali@kali)-[~/Desktop]
$ gcc -E hello.c -o hello.i

(kali@kali)-[~/Desktop]
$ gcc -S hello.i -o hello.s

(kali@kali)-[~/Desktop]
$ gcc -c hello.s -o hello.o

(kali@kali)-[~/Desktop]
$ gcc hello.o -o hello

(kali@kali)-[~/Desktop]
$ ls -l
total 256
-rw-rw-r-- 1 kali kali 103 Sep 7 13:41 aayush.c
drwxrwxr-x 2 kali kali 4096 Sep 7 00:21 folders
-rw-rw-r-- 1 kali kali 448 Sep 20 11:05 formattedoutput.c
-rwxrwxr-x 1 kali kali 15952 Sep 20 12:09 hello
-rw-rw-r-- 1 kali kali 72 Sep 20 12:06 hello.c
-rw-rw-r-- 1 kali kali 4003 Sep 7 00:54 hello.c.save
-rw-rw-r-- 1 kali kali 21424 Sep 20 12:09 hello.i
-rw-rw-r-- 1 kali kali 1376 Sep 20 12:09 hello.o
-rw-rw-r-- 1 kali kali 489 Sep 20 12:09 hello.s
-rw-rw-r-- 1 kali kali 1592 Sep 15 02:18 input.c

```

5) Task 5

Compiled and ran a program returning the integer value 50.

```

Session Actions Edit View Help
GNU nano 9.2 exit50.c *
#include <stdio.h>

int main() {
    printf("Returning exit code 50\n");
    return 50;
}

```

```

(kali@kali)-[~/Desktop]
$ nano exit50.c

(kali@kali)-[~/Desktop]
$ gcc exit50.c -o exit51

(kali@kali)-[~/Desktop]
$ ./exit51
Returning exit code 50

```

VII. Conclusion

This lab exercise provided hands-on experience with C programming fundamentals, including program structure, compilation process, and GCC usage. Understanding these concepts is essential for systems programming and operating system development. The four-stage compilation process demonstrates how high-level code is transformed into machine-executable instructions.

GitHub Repository

All source code, compilation outputs, and this report have been submitted to the course GitHub repository.

Github repo link : [Rupeshycho/240565_Rupesh_Yadav_linux](https://github.com/Rupeshycho/240565_Rupesh_Yadav_linux)